I generally favor `GetRequiredService` over `GetService`, as it immediately tells you whether you have a configuration problem with your DI container by throwing an exception, and you don't have to handle `null`s.

You can use either of these methods in Program.cs to retrieve a service from DI. The following listing shows how to retrieve a `LinkGenerator` from the DI container, but you can access any service registered in the DI container here.

Listing 8.8 Retrieving a service from the DI container using `WebApplication.Services`

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/", () => "Hello World!");

LinkGenerator links =                                          ❶
    app.Services.GetRequiredService<LinkGenerator>();          ❶

app.Run();                                                     ❷
```

❶ Retrieves a service from the DI container using the GetRequiredService<T>() extension method

❷ You must access services before app.Run(), as this call blocks until your app exits.

This approach, in which you call the DI container directly to ask for a class, is called the *service locator* pattern. Generally speaking, you should try to avoid this pattern in your code; include your dependencies as constructor or endpoint handler arguments directly, and let the DI container provide them for you. This pattern is the only way to access DI services in the main loop of your application in Program.cs, however, so don't worry about using it here. Still, you should absolutely avoid accessing `WebApplication.Services` from inside your endpoint handlers or other types whenever possible.